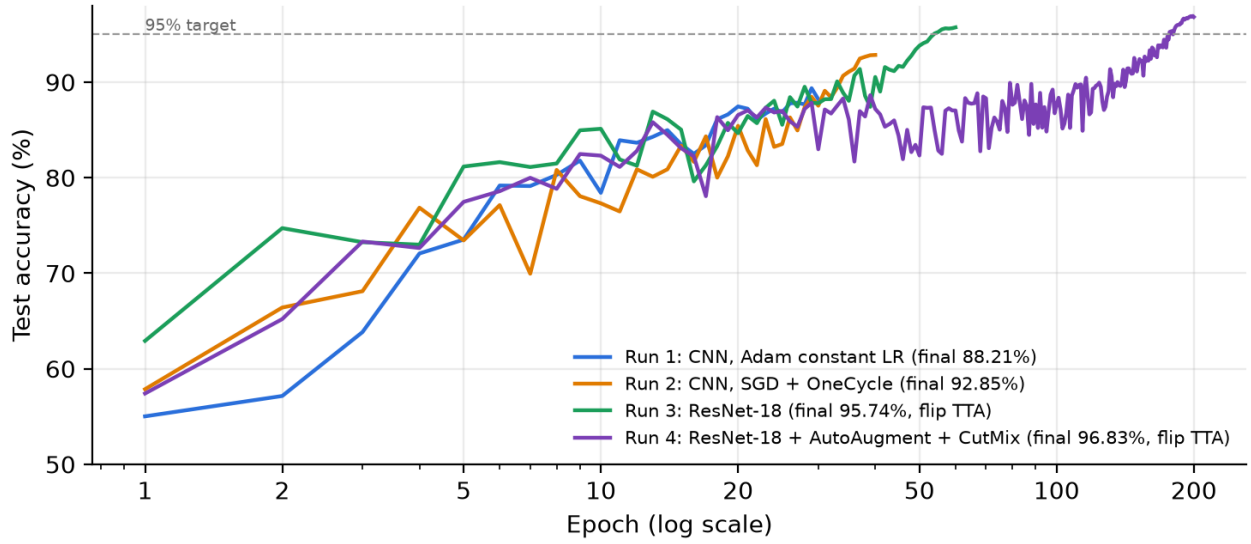
CIFAR-10 CNN: Experiment Log

Deep Learning Assignment 2: what was tried, what changed between runs, why each change was chosen, and the results.

Summary

Run	Model	Training setup	Epochs	Final test acc	Best test acc
1	6-conv VGG-style CNN, FC head (4096→256→10)	Adam lr=1e-3 constant, RandomResizedCrop + flip, plain cross-entropy	30	**88.21%**	89.37% (ep 29)
2	Same convs, global-average-pool head	SGD + Nesterov, wd=5e-4, OneCycleLR (max 0.1), RandomCrop(pad 4) + flip + RandomErasing, label smoothing 0.1	40	**92.85%**	92.85% (ep 40)
3	ResNet-18 (11.2M params)	Run 2 recipe + bf16 mixed precision + flip test-time augmentation	60	**95.74%** (TTA) 95.15% (no TTA)	95.74% (ep 60)
4	ResNet-18	Run 3 + AutoAugment (CIFAR-10 policy) + CutMix (p=0.5)	200	**96.83%** (TTA) 96.53% (no TTA)	96.93% (ep 199)
5	ViT-B/16, pretrained (SWAG, 86M params)	Transfer learning: fine-tune at 224px, AdamW lr 5e-5, OneCycle, label smoothing, flip TTA	10	*pending*	*pending*

Current best: 96.83% test accuracy (Run 4, ResNet-18 trained from scratch with AutoAugment, CutMix and flip TTA), 8.62 percentage points above the Run 1 baseline. Run 5 (pretrained ViT, target 98%+) is set up but hasn't been run in full yet.



Run 1: Baseline CNN

Configuration

- **Model:** 3 blocks of [Conv3x3 → BN → ReLU] × 2 + MaxPool, channels 64/128/256 (32×32 → 4×4). Classifier: Flatten → Linear(4096, 256) → ReLU → Dropout(0.4) → Linear(256, 10).

- **Data:** RandomHorizontalFlip, RandomResizedCrop(32, scale 0.7–1.0), per-channel normalization. Batch size 128, num_workers=0.
- **Optimization:** Adam, lr=1e-3 (constant, no schedule), CrossEntropyLoss, 30 epochs.

Result

Final test accuracy **88.21%**, best 89.37% at epoch 29. Test accuracy kept fluctuating up to the last epoch (89.37% → 88.21% over epochs 29–30), because the learning rate never decays and the weights never settle into a minimum. Train loss (0.22) was well below test loss (0.38), so the model was starting to overfit.

Changes after Run 1, and why

Change	Why it was chosen
Adam (constant lr) → **SGD**, momentum 0.9, Nesterov, **weight decay 5e-4**	SGD with weight decay usually generalizes better than Adam on CIFAR. Weight decay is L2 regularization.
Added **OneCycleLR** (max_lr 0.1), stepped every batch; epochs 30 → **40**	Warms the LR up, then anneals it to almost zero, so training ends at a stable minimum instead of bouncing around. This was the biggest expected gain.
RandomResizedCrop → **RandomCrop(32, padding=4, reflect)** + flip; added **RandomErasing(p=0.25)**	Padded crop is the standard CIFAR augmentation and keeps the object's scale the same. Random erasing adds regularization.
CrossEntropyLoss → **label_smoothing=0.1**	Reduces overconfidence. It also means loss can't drop below about 0.5, so higher loss values are expected.
Flatten + Linear(4096,256) + Dropout(0.4) → **AdaptiveAvgPool** → **Dropout(0.3)** → **Linear(256,10)**	Removes about 1M FC parameters, which were the main source of overfitting.

Speed-only changes (no effect on accuracy): GPU utilization was about 30% because data loading ran on one CPU process. DataLoaders were switched to num_workers=4, pin_memory=True, persistent_workers=True, and they're re-created *after* the augmentation transforms are set (persistent workers keep their own copy of the dataset, so loaders created earlier would keep the old transform).

Note: an intermediate 30-epoch run that finished at 88.53% was recorded partway through these edits. Its exact configuration wasn't saved (its train loss of 0.22 shows label smoothing wasn't active yet), so it isn't counted as a separate experiment here.

Run 2: Improved training recipe (CNN)

Final test accuracy **92.85%**, which is also the best epoch. Between epochs 5–25, accuracy jumped around (e.g. 80.81% → 77.32% → 76.47%). That's expected with OneCycle: the LR peaks at 0.1 around epoch 12, and large steps act as regularization. Once the LR anneals, accuracy rises almost every epoch (89.10% → 92.85% over epochs 31–40). Train loss (0.66) and test loss (0.71) stay close, so overfitting is much lower than in Run 1.

Changes after Run 2, and why

The goal was 95%+. A plain 6-conv CNN tops out around 92–93% on CIFAR-10 even with a strong recipe, so the training recipe was kept and the architecture was replaced. Run 3 was trained in a separate notebook, assignment_2_resnet.ipynb.

Change	Why it was chosen
Plain CNN → **ResNet-18** (3×3 stem, 4 stages of 2 BasicBlocks, 64/128/256/512 channels, global avg pool, Linear(512,10))	Skip connections let gradients flow through 18 layers, so the network can be deeper and more accurate.
bf16 mixed precision (torch.autocast) + channels_last	ResNet-18 is about 10× bigger. Mixed precision keeps epochs fast (about 20 s on an RTX 4080 SUPER).
Flip test-time augmentation	Averaging predictions on each image and its mirror image is a cheap accuracy gain at test time.
Epochs 40 → **60**	A bigger model needs more epochs to converge.

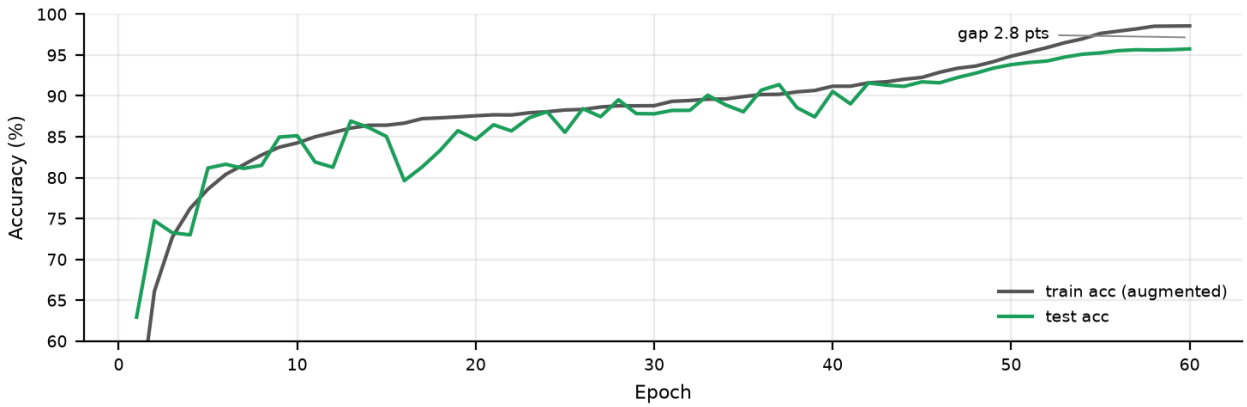
Run 3: ResNet-18

Final test accuracy **95.74%** with flip TTA and **95.15%** without it, so TTA adds 0.59 points. That passes the 95% target. The same OneCycle pattern appears: noisy between epochs 10–40, then steady climbing (90.54% at epoch 40 → 95.74% at epoch 60).

Per-class test accuracy (with TTA):

Class	airplane	automobile	bird	cat	deer
Acc	95.70%	98.30%	94.10%	90.90%	97.00%
Class	dog	frog	horse	ship	truck
Acc	92.40%	98.00%	96.70%	97.80%	96.50%

What the results showed



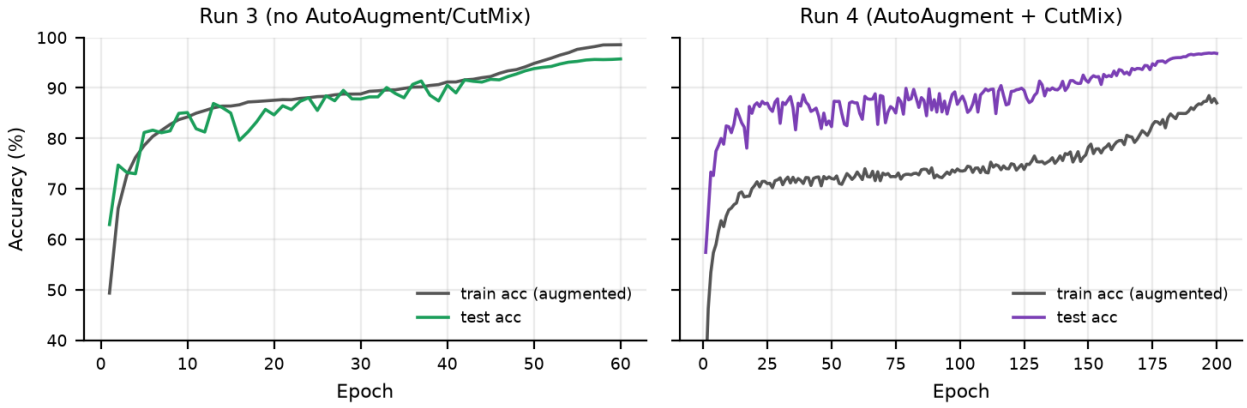
- **Still improving at the end:** accuracy rose almost every epoch up to epoch 60, so a longer schedule should help.
- **Overfitting:** train accuracy reached 98.56% even on augmented images, against 95.74% test. With 11M parameters the model memorizes the training set, so stronger augmentation should help more than a bigger model.
- **Weakest classes:** cat (90.9%) and dog (92.4%), which look alike. Richer augmentation is the usual fix.

Changes after Run 3, and why

Change	Why it was chosen
Added **AutoAugment** (torchvision CIFAR-10 policy) after the flip	A learned policy of colour, contrast, shear and rotation changes. It's the most effective single fix for the overfitting gap. Expected about +0.8%.
Added **CutMix** on 50% of batches	Pastes a patch from one image into another and mixes the labels in proportion to the patch size. This adds regularization and helps confusable classes like cat and dog. Expected about +0.3–0.5%.
Epochs 60 → **200**	Run 3 hadn't finished improving, and stronger augmentation needs more epochs to pay off. Takes about 1 hour.
num_workers 4 → **8**	AutoAugment runs on the CPU. More workers stop it from slowing the GPU down.

Run 4: ResNet-18 + AutoAugment + CutMix

Final test accuracy **96.83%** with flip TTA and **96.53%** without it, +1.09 points over Run 3 (expected about 96.5–97%). The best epoch was 199 (96.93%), and the result is the same model at epoch 200. The stronger augmentation worked as intended: train accuracy on augmented data is now *below* test accuracy (87.02% vs 96.83%), so the model no longer memorizes the training set. Test accuracy stayed noisy and low (around 82–91%) until the LR annealed after about epoch 150, then rose steadily.



Weakest classes improved: cat 90.90% → 91.70%, dog 92.40% → 94.70%. Note: the Run 3 weights file was overwritten by Run 4. Run 3's results are preserved in assignment_2_resnet_run3.ipynb.

Changes after Run 4, and why (Run 5, pending)

The new target was 98%+. Training from scratch on 50,000 images tops out around 97–98% even with much bigger networks (WideResNet-28-10 reaches about 97.5% after several hours), so the remaining gain had to come from **transfer learning**: reusing features learned from a far larger dataset. This is a different approach from Runs 1–4, which all trained from scratch, and it's in a separate notebook, assignment_2_vit.ipynb.

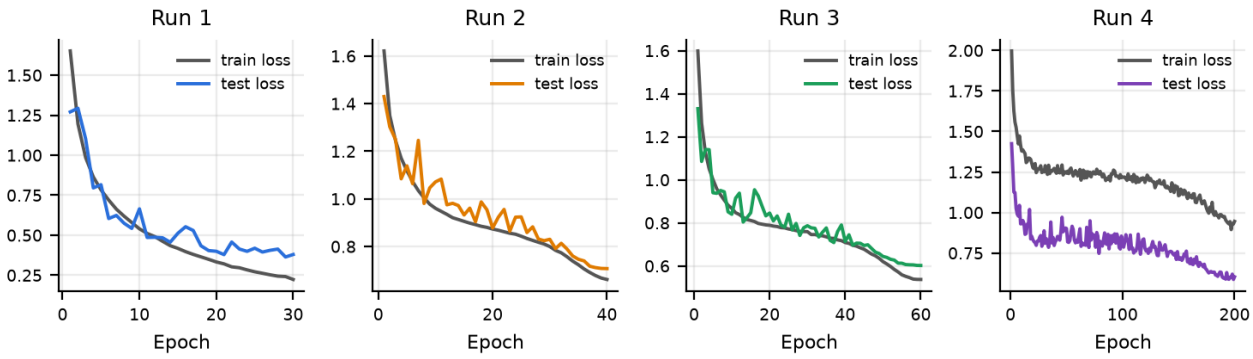
Change	Why it was chosen
ResNet-18 from scratch → **ViT-B/16 pretrained** (torchvision SWAG weights: 3.6B Instagram images, then ImageNet), new zero-initialized 10-class head	Strong pretrained features make CIFAR-10 an easy downstream task. Available in torchvision with no extra packages.
Images upscaled **32** → **224 px** on the GPU after augmentation; ImageNet normalization	The model was pretrained at 224 px with ImageNet statistics. Upscaling on the GPU keeps the CPU from becoming a bottleneck.
AdamW, lr 5e-5, weight decay 0.05, OneCycle (10% warm-up), gradient clipping 1.0, batch 64	Fine-tuning needs a small LR so the pretrained weights only get adjusted, not overwritten. AdamW is the standard optimizer for transformers.
10 epochs; dropped AutoAugment, CutMix and RandomErasing (kept crop + flip, label smoothing, flip TTA)	A pretrained model converges in a few epochs, and heavy augmentation mainly helps long from-scratch training. About 2.8 min/epoch, about 30 min total.

Sanity check: a 1-epoch test run of this notebook already reached **97.08%** (96.74% without TTA), higher than Run 4 after 200 epochs. The full 10-epoch run is expected to reach about 98.5%+.

Options considered but not chosen

- **WideResNet-28-10 from scratch** (about 97.5%): several hours of training and still unlikely to reach 98%.
- **Weight EMA** (+0.1–0.3%) and **ensembling** several models (+0.5% or more): not needed if fine-tuning clears 98%.

Loss curves



Runs 2–4 use label smoothing, so their losses can't go below about 0.5.

Methodology note

All accuracies above are on the CIFAR-10 test set, and the same test set was also used to compare configurations, so the final number is slightly optimistic. Run 5 also uses outside data (pretrained weights), so it should be reported as transfer learning, separately from the from-scratch results. For a fully unbiased final number, a validation split should be held out from the 50,000 training images for tuning, with the test set evaluated only once at the end.